

Artifact for RTSS 2026

This repository contains the source code, datasets, and precomputed results supporting the paper "*Real-Time Multi-Telescope Scheduling to Search for Transient Astrophysical Phenomena*."

- Source repository: [RTSS26-Multi-Telescope-Transient-Follow-Up-Search](#)

Table of Contents

- [Artifact Scope](#)
 - [Implemented Methods](#)
 - [Supported Artifact Claims](#)
- [System Requirements](#)
 - [Software](#)
 - [Hardware and Storage](#)
- [Repository Structure](#)
- [1. Environment Setup](#)
 - [1.1 Obtaining a Gurobi WLS License](#)
 - [1.2 \(Option A, Preferred\) Using the Provided Docker Container](#)
 - [1.2.1 Install Docker](#)
 - [1.2.2 Pull the Docker Image](#)
 - [1.3 \(Option B\) Local Installation](#)
 - [1.3.1 Clone the Repository](#)
 - [1.3.2 Python Environment Setup](#)
 - [1.3.3 C++ Environment Setup](#)
- [2. Reproducing the Paper Figures from Precomputed Results](#)
 - [2.1 Option A: Run JupyterLab in the Docker Container](#)
 - [2.2 Option B: Run JupyterLab Locally](#)
 - [2.3 Notebook Outputs](#)
 - [2.4 Optional Interactive Route Visualizations](#)
- [3. Re-running the Paper Experiments \(Long-Running\)](#)
 - [3.1 Experiment Script Setup](#)
 - [3.2 Common Evaluation Configuration](#)
 - [3.3 Run All Experiments](#)
 - [3.3.1 Fixed- \$K\$ Cross-Instance Experiments \(Figures 2-4\)](#)
 - [3.3.2 Fixed- \$K\$ Deadline Sweeps \(Figures 5 and 6\)](#)
 - [3.3.3 Fixed- \$K\$ Telescope-Count Sweep \(Figure 7\)](#)
 - [3.3.4 Minimum-Demand Cross-Instance Experiments \(Figures 8-10\)](#)
 - [3.3.5 Minimum-Demand Deadline Sweep \(Figure 11\)](#)
 - [3.3.6 Run Script Summary: Run All Experiments](#)
 - [3.4 Visualize Reviewer-Generated Results](#)
 - [3.5 Optional: Reduced or Partial Evaluations](#)
 - [3.5.1 Maximum-Probability Experiments \(\$ts_{maxp}\$ \)](#)
 - [3.5.2 Minimum-Telescope Experiments \(\$ts_{mink}\$ \)](#)
 - [3.5.3 Notes for All Reduced Runs](#)

- [4. Extending the Artifact](#)
 - [4.1 Run a Single Instance](#)
 - [4.2 Generate a Different Non-Overlapping Tiling](#)
 - [4.3 Explore Different Slew Parameters and Separation Models](#)
 - [4.4 Explore Different Dwell-Time Parameters](#)

Artifact Scope

The paper studies two complementary deadline-constrained scheduling objectives for multi-telescope follow-up search:

1. **Fixed-resource scheduling (maximum detection probability):** Given K telescopes and a route deadline D , select one route per telescope to maximize the total, non-duplicated detection probability covered before the deadline.
2. **Resource provisioning (minimum telescope demand):** Given a route deadline D , cover all candidate tiles while minimizing the number of rooted telescope routes.

The artifact contains implementations of the practical algorithms and ILP baselines evaluated in the paper, data derived from real LIGO localization maps, precomputed experimental outputs, and notebooks that generate the paper figures.

Implemented Methods

Objective	Method	Description	Requires a Gurobi license at runtime?
Maximum detection probability	GREEDY+SA	Residual-greedy multi-telescope scheduling with a simulated-annealing single-route planner	No
Maximum detection probability	GREEDY+ILP	Residual-greedy scheduling with a Gurobi single-route orienteering planner	Yes
Maximum detection probability	TOP-SA	Simulated annealing over the joint team-orienteering solution	No
Maximum detection probability	TOP-ILP	Gurobi formulation of the full team-orienteering problem; reports a feasible solution and an upper bound	Yes
Minimum telescope demand	R-GREEDY	Rooted component decomposition and MST-based tour splitting	No
Minimum telescope demand	R-SA	Repeated rooted single-route simulated annealing until all tiles are covered	No

Objective	Method	Description	Requires a Gurobi license at runtime?
Minimum telescope demand	R-ILP	Gurobi formulation of the full minimum-route problem; reports a feasible solution and a lower bound	Yes

Supported Artifact Claims

The artifact is intended to support the following activities:

- Reproduce Figures 2-11 from the provided precomputed results.
- Re-run the fixed-K maximum-probability experiments.
- Re-run the minimum-telescope-demand experiments.
- Visualize the telescope routes produced by the algorithms.
- Run a selected localization-map instance through either objective driver.

System Requirements

Software

- Linux operating system. The supplied installation commands assume an x86-64 Ubuntu system or a compatible distribution.
- A C++17 compiler.
- CMake 3.13 or later.
- Gurobi Optimizer 12.0.x. Gurobi headers and libraries are required to compile the complete project, and a valid license is required to execute the ILP-based methods.
- LEMON Graph Library 1.3.1.
- Python 3.11 for experiment drivers, notebook execution, and visualization.
- Python dependencies are specified in `environment.yml`, which defines the `rtss26-figures` conda environment.

Hardware and Storage

- Recommended CPU: 8 or more cores; the paper jobs requested 36 cores.
- Minimum RAM: 36 GB for setup, plotting, and small smoke tests.
- Recommended RAM for complete experiments: 64 GB for small instances; the large-instance jobs in the paper requested up to 1+ TB.
- Free storage required for a local installation: approximately 10 GB.
- Published container image size: approximately 3.5 GB. Allow at least 5 GB of free storage for the image and Docker's temporary layers; 10 GB is recommended when retaining generated results and figures.
- GPU: Not required.

For reference, the paper experiments used the following shared Slurm nodes:

- Small instances: AMD EPYC 7713 nodes with two 64-core sockets and 512 GB RAM; each job requested 36 cores and 64 GB RAM.

- Large instances: AMD EPYC 9654 nodes with two 96-core sockets and 1.54 TB RAM; each job requested 36 cores and 1.5 TB RAM.
- HyperThreading was disabled, and no GPUs were used.

These are the paper's execution platforms, not necessarily minimum artifact requirements. Full ILP experiments can require substantial time and memory.

Repository Structure

```
.
|-- data/                                # Input localization-map and tiling
data
|   |-- generate_sky_tiling/              # Small maps used in the paper
|   |-- small_maps_6.9x6.9_tiling/        # Large maps used in the paper
|   |-- large_maps_4.0x2.0_tiling/        # Very large maps used in the paper
|   |-- very_large_maps_1.34x0.9_tiling/  # Sky-tiling generator
|   `-- tilings/                          # Tilings used by the experiments
|-- environment.yml                       # Python environment
|-- include/                              # Header files for planners
|   |-- apps/                             # Application
|   |-- common/                           # Shared data structures and
utilities
|   |-- multi_telescope/                  # Multi-telescope algorithms
|   `-- single_telescope/                 # Single-telescope algorithms
|-- precompute_results/
|   |-- RTSS2026_Paper_Figures.ipynb      # Figures 2-11 in manuscript order
|   |-- max_probability/                  # Precomputed fixed-K results
|   `-- min_telescope/                    # Precomputed minimum-demand results
|-- results/                              # Experiment scripts and new outputs
|   |-- RTSS2026_Rerun_Results.ipynb      # Figures from reviewer-generated
CSVs
|   |-- run_maxp_sweep_maps.py             # Figures 2-4 experiment script
|   |-- run_maxp_sweep_deadlines.py        # Figures 5-6 experiment script
|   |-- run_maxp_sweep_npaths.py           # Figure 7 experiment script
|   |-- run_mink_small.py                  # Small-map minimum-demand script
|   |-- run_mink_large.py                  # Large-map minimum-demand script
|   `-- run_mink_sweep_deadlines.py        # Figure 11 experiment script
|-- src/
|   |-- apps/                             # Application/CLI implementations
|   |-- common/                           # Shared implementations
|   |-- multi_telescope/                  # Multi-telescope implementations
|   |-- single_telescope/                 # Single-route implementations
|   |-- main.cpp
|   |-- main_maxp_entry.cpp                # Entry point for ts_maxp
|   `-- main_mink_entry.cpp                # Entry point for ts_mink
|-- CMakeLists.txt                        # CMake build configuration
`-- README.md                             # This file
```

1. Environment Setup

You can run the artifact via **Docker (recommended)** or a **local setup**. A Gurobi license is needed only to run ILP-based experiments.

1.1 Obtaining a Gurobi WLS License

Our algorithms include ILP implementations of the orienteering problem, which are solved using Gurobi Optimizer. This artifact documents the **Web License Service (WLS)** configuration for both Docker and local execution.

Gurobi requires a license file named `gurobi.lic` to run the ILP-based methods.

- Refer to [How to Retrieve and Set Up a Gurobi License](#).
- Without a valid license, the `simulated-annealing` and `R-GREEDY` methods can still run. However, the Gurobi headers and libraries are required at build time.
- WLS requires an Internet connection while Gurobi is in use.

Academic users can obtain a free Academic WLS license:

1. Navigate to the [Gurobi portal](#).
2. Log in or register using your academic email address.
3. Navigate to the [academic license request page](#).
4. Under **WLS Academic**, click **Generate Now**.
5. Create an API key and download the generated `gurobi.lic` file.
6. Place the file at `$HOME/gurobi.lic`, which is Gurobi's default user-level location, or keep it in another location and set `GRB_LICENSE_FILE` to its complete path.

The following commands use `$HOME/gurobi.lic` by default:

```
export GRB_LICENSE_FILE="${GRB_LICENSE_FILE:-$HOME/gurobi.lic}"

# To use another location instead:
# export GRB_LICENSE_FILE="/absolute/path/to/gurobi.lic"

chmod 600 "$GRB_LICENSE_FILE"
```

1.2 (Option A, Preferred) Using the Provided Docker Container

1.2.1 Install Docker

You may install Docker according to [these instructions](#). Here, we include the instructions for Ubuntu distributions.

1. Set up Docker's `apt` repository:

```
# Add Docker's official GPG key:
sudo apt-get update
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Add the repository to Apt sources:
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
  by=/etc/apt/keyrings/docker.asc] \
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
  stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

2. Install the latest Docker packages:

```
sudo apt-get install docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

1.2.2 Pull the Docker Image

```
export RTSS_IMAGE="${RTSS_IMAGE:-ghcr.io/daisy0419/rtss26-multi-
telescope:1.0}"
sudo docker pull "$RTSS_IMAGE"
```

All dependencies are pre-installed, and project binaries are precompiled in the image. You can jump to Section 2 or 3 to visualize the figures or run full experiments using container.

1.3 (Option B) Local Installation

1.3.1 Clone the Repository

Clone the repository to the directory of your choice. The following commands use `$HOME/RTSS26-Multi-Telescope-Transient-Follow-Up-Search` by default:

```
export PROJECT_DIR="${PROJECT_DIR:-$HOME/RTSS26-Multi-Telescope-Transient-
Follow-Up-Search}"

# To use another location instead:
# export PROJECT_DIR="/path/to/RTSS26-Multi-Telescope-Transient-Follow-Up-
Search"

git clone \
  https://github.com/Daisy0419/RTSS26-Multi-Telescope-Transient-Follow-Up-
Search.git \
  "$PROJECT_DIR"
```

```
cd "$PROJECT_DIR"
```

1.3.2 Python Environment Setup

We recommend setting up a [conda](#) environment for Python.

If conda is not installed locally, download and install Miniconda. The following commands use `$HOME/conda` by default:

```
export PROJECT_DIR="${PROJECT_DIR:-$HOME/RTSS26-Multi-Telescope-Transient-
Follow-Up-Search}"
export CONDA_DIR="${CONDA_DIR:-$HOME/conda}"

# To use another installation location:
# export CONDA_DIR="/path/to/conda"

cd "$PROJECT_DIR"
wget -q \
  https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh \
  -O miniconda.sh
bash miniconda.sh -b -p "$CONDA_DIR"
rm miniconda.sh
```

Make conda available in the current shell:

```
. "$CONDA_DIR/etc/profile.d/conda.sh"
conda config --system --set channel_priority flexible
```

Once conda is available, create the artifact's Python environment from the provided YAML file:

```
cd "$PROJECT_DIR"
conda env create -f environment.yml
```

Activate it before running the experiment scripts, paper-figure notebook, or route-visualization utility:

```
conda activate rtss26-figures
```

The environment installs Python 3.11 together with NumPy, pandas, Matplotlib, Plotly, JupyterLab, the Jupyter execution and export packages, and Pillow.

1.3.3 C++ Environment Setup

(1) Gurobi Optimizer (Required)

Gurobi Optimizer is used by the ILP-based planning methods. The artifact was developed with Gurobi 12.0.1.

1. Choose a directory in which to install Gurobi. The following commands use `$HOME` by default:

```
export GUROBI_BASE_DIR="${GUROBI_BASE_DIR:-$HOME}"

# To use another location instead:
# export GUROBI_BASE_DIR="/path/to/designated/directory"

mkdir -p "$GUROBI_BASE_DIR"
cd "$GUROBI_BASE_DIR"

wget https://packages.gurobi.com/12.0/gurobi12.0.1_linux64.tar.gz
tar xvfz gurobi12.0.1_linux64.tar.gz
```

The extracted installation is located at:

```
$GUROBI_BASE_DIR/gurobi1201/linux64
```

2. Set the Gurobi environment variables:

```
export GUROBI_BASE_DIR="${GUROBI_BASE_DIR:-$HOME}"
export GUROBI_HOME="$GUROBI_BASE_DIR/gurobi1201/linux64"
export PATH="${GUROBI_HOME}/bin:${PATH}"
export LD_LIBRARY_PATH="${GUROBI_HOME}/lib:${LD_LIBRARY_PATH:-}"
```

Note: Please repeat these commands in each new shell or add them to the appropriate shell startup file, such as `~/.bashrc`.

Verify the Gurobi installation:

```
"$GUROBI_HOME/bin/gurobi_cl" --version
```

3. Configure and test the WLS license.

Suppose the license is stored at `$HOME/gurobi.lic`, run:

```
export GRB_LICENSE_FILE="${GRB_LICENSE_FILE:-$HOME/gurobi.lic}"
# If the license is stored elsewhere, set its complete path instead:
# export GRB_LICENSE_FILE="/absolute/path/to/gurobi.lic"
```



```
chmod 600 "$GRB_LICENSE_FILE"
"$GUROBI_HOME/bin/gurobi_cl" --license
```

Gurobi reads the API credentials from `gurobi.lic` and obtains a WLS token when a Gurobi environment is created.

A successful test reports the license being used and does not display a license error. Add the `GRB_LICENSE_FILE` export to your shell startup file if the license is stored outside Gurobi's default locations.

Note: Gurobi must be installed to compile this artifact because its headers and libraries are linked at build time. A valid WLS license is additionally required when running any method that creates a Gurobi environment. The supplied experiment scripts execute the ILP methods, so rerunning the paper experiments requires a working WLS license. Each reviewer must provide their own license; no license file or WLS credential is distributed with the artifact.

(2) LEMON Graph Library (Required)

LEMON provides graph data structures and matching routines used by the retained single-route planning code.

1. Choose a directory in which to install LEMON. The following commands use `$HOME` by default:

```
export LEMON_BASE_DIR="${LEMON_BASE_DIR:-$HOME}"

# To use another location instead:
# export LEMON_BASE_DIR="/path/to/designated/directory"

mkdir -p "$LEMON_BASE_DIR"
cd "$LEMON_BASE_DIR"

wget http://lemon.cs.elte.hu/pub/sources/lemon-1.3.1.tar.gz
tar xvfz lemon-1.3.1.tar.gz

cmake -S "$LEMON_BASE_DIR/lemon-1.3.1" \
      -B "$LEMON_BASE_DIR/lemon-1.3.1/build"

cmake --build "$LEMON_BASE_DIR/lemon-1.3.1/build" --parallel
```

2. Pass the source and build directories when configuring the artifact, as shown in the next section:

```
cmake -S "$PROJECT_DIR" \
      -B "$PROJECT_DIR/build" \
      -DLEMON_SOURCE_DIR="$LEMON_BASE_DIR/lemon-1.3.1" \
      -DLEMON_BUILD_DIR="$LEMON_BASE_DIR/lemon-1.3.1/build"
```

(3) Build the C++ Executables

Once all dependencies are installed, build the C++ project with:

```
export PROJECT_DIR="${PROJECT_DIR:-$HOME/RTSS26-Multi-Telescope-Transient-Follow-Up-Search}"
export LEMON_BASE_DIR="${LEMON_BASE_DIR:-$HOME}"

# To use other locations instead:
# export PROJECT_DIR="/path/to/RTSS26-Multi-Telescope-Transient-Follow-Up-Search"
# export LEMON_BASE_DIR="/path/to/designated/directory"

cmake -S "$PROJECT_DIR" \
  -B "$PROJECT_DIR/build" \
  -DLEMON_SOURCE_DIR="$LEMON_BASE_DIR/lemon-1.3.1" \
  -DLEMON_BUILD_DIR="$LEMON_BASE_DIR/lemon-1.3.1/build"

cmake --build "$PROJECT_DIR/build" --parallel
```

This produces two binaries in `build/`:

- `ts_maxp` — fixed- K maximum-detection-probability experiments.
- `ts_mink` — minimum-telescope-demand experiments.

2. Reproducing the Paper Figures from Precomputed Results

Precomputed results are provided so that an evaluator can reproduce the paper figures without rerunning the long ILP experiments.

2.1 Option A: Run JupyterLab in the Docker Container

This option uses the Python environment packaged in the published artifact container; it does not require a local conda installation or a Gurobi license.

From the repository root, run:

```
sudo docker run --rm -it -p 127.0.0.1:8888:8888 \
  -v "$PWD:/workspace" \
  ghcr.io/daisy0419/rtss26-multi-telescope:1.0 \
  bash -lc 'conda run --no-capture-output -n rtss26-figures \
    jupyter lab --ip=0.0.0.0 --port=8888 --no-browser \
      --IdentityProvider.token="" \
      --ServerApp.root_dir=/workspace \
      --allow-root'
```

Then open <http://localhost:8888> and navigate to `precompute_results/RTSS2026_Paper_Figures.ipynb` in the sidebar.

2.2 Option B: Run JupyterLab Locally

Complete the local installation in Section 1.3, then run JupyterLab from the repository root.

The commands below use `$HOME/RTSS26-Multi-Telescope-Transient-Follow-Up-Search` by default. If the repository is stored elsewhere, set `RTSS_REPO` to that location instead:

```
export RTSS_REPO="{RTSS_REPO:-$HOME/RTSS26-Multi-Telescope-Transient-
Follow-Up-Search}"

# For another location, use this instead:
# export RTSS_REPO="/path/to/RTSS26-Multi-Telescope-Transient-Follow-Up-
Search"
```

```
cd "$RTSS_REPO"
conda activate rtss26-figures
jupyter lab precompute_results/RTSS2026_Paper_Figures.ipynb
```

2.3 Notebook Outputs

`precompute_results/RTSS2026_Paper_Figures.ipynb` loads the precomputed CSV and map data and generates Figures 2-11 in manuscript order. It does not rerun an optimizer and does not require a Gurobi license.

Publication-ready PDFs and 300-DPI PNGs are saved under `precompute_results/paper_figures/` with these stems:

```
fig02_maxp_detection_probability
fig03_maxp_planning_time
fig04_maxp_routes_200220
fig05_maxp_deadline_191113
fig06_maxp_deadline_200105_11678
fig07_maxp_telescope_count_191113
fig08_mink_route_count
fig09_mink_planning_time
fig10_mink_routes_200216
fig11_mink_deadline_191113
```

Each stem is written as both `.pdf` and `.png`. Figure generation is expected to complete in under five minutes.

2.4 Optional Interactive Route Visualizations

Figures 4 and 10 are saved as static paper figures by default and it may not align the view as presented in the paper. To inspect the routes interactively from different view, set `RUN_INTERACTIVE = True` in the notebook's configuration cell, then rerun the Figure 4 and Figure 10 cells. The viewers support zooming, panning, rotation, and per-route hover information.

3. Re-running the Paper Experiments (Long-Running)

Full re-execution is substantially more expensive than plotting the precomputed results and may require approximately **50 hours or more**, depending on the platform and how often the ILP baselines reach their limits. A valid Gurobi license is required because the current experiment scripts execute the ILP methods as well as the simulated-annealing and greedy methods.

Optional Section 3.5, at the end of this section, provides shorter reviewer configurations. These reduced runs are intended to check the end-to-end execution of the artifact, not to reproduce the paper's numerical results. Please skip Sections 3.2-3.4 and jump to Section 3.5 (after completing the setup in Section 3.1) if you do not have enough time to run full experiments.

3.1 Experiment Script Setup

The six experiment scripts are stored under `results/`. Use either the Docker setup below or the local setup in Section 1.3.

Docker setup. The commands below copy the writable `results/` directory from the image into a host work directory and then mount it back into the container. This lets reviewers edit script configurations and retain generated CSV and figure files after the container exits. The default host location is `$HOME/rtss26-artifact`, but it may be changed to any writable directory.

```
export RTSS_IMAGE="${RTSS_IMAGE:-ghcr.io/daisy0419/rtss26-multi-
telescope:1.0}"
export RTSS_CONTAINER_REPO="/opt/RTSS26-Multi-Telescope-Transient-Follow-
Up-Search"
export RTSS_WORK_DIR="${RTSS_WORK_DIR:-$HOME/rtss26-artifact}"
export GRB_LICENSE_FILE="${GRB_LICENSE_FILE:-$HOME/gurobi.lic}"

# To use other host locations instead:
# export RTSS_WORK_DIR="/path/to/rtss26-artifact"
# export GRB_LICENSE_FILE="/absolute/path/to/gurobi.lic"

mkdir -p "$RTSS_WORK_DIR"

# Copy the editable experiment directory from the image only once.
if [ ! -d "$RTSS_WORK_DIR/results" ]; then
  RTSS_COPY_CONTAINER="$(sudo docker create "$RTSS_IMAGE")"
  sudo docker cp \
    "$RTSS_COPY_CONTAINER:$RTSS_CONTAINER_REPO/results" \
    "$RTSS_WORK_DIR/"
  sudo docker rm "$RTSS_COPY_CONTAINER"
  sudo chown -R "$(id -u):$(id -g)" "$RTSS_WORK_DIR/results"
fi

test -f "$GRB_LICENSE_FILE"
export GRB_LICENSE_FILE="$(realpath "$GRB_LICENSE_FILE")"
chmod 600 "$GRB_LICENSE_FILE"

# Verify the reviewer-provided WLS license before starting long runs.
sudo docker run --rm \
```

```

--mount
type=bind,source="$GRB_LICENSE_FILE",target=/opt/gurobi/gurobi.lic,readonly
\
--env GRB_LICENSE_FILE=/opt/gurobi/gurobi.lic \
"$RTSS_IMAGE" \
gurobi_cl --license

sudo docker run --rm -it \
--user "$(id -u):$(id -g)" \
--env HOME=/tmp \
--env USER=rtss-reviewer \
--env RTSS_REPO="$RTSS_CONTAINER_REPO" \
--mount
type=bind,source="$GRB_LICENSE_FILE",target=/opt/gurobi/gurobi.lic,readonly
\
--env GRB_LICENSE_FILE=/opt/gurobi/gurobi.lic \
-p 127.0.0.1:8888:8888 \
--mount
type=bind,source="$RTSS_WORK_DIR/results",target="$RTSS_CONTAINER_REPO/resu
lts" \
--workdir "$RTSS_CONTAINER_REPO" \
"$RTSS_IMAGE" \
bash

```

Run the commands in Section 3.3 from the resulting container shell. Type `exit` when finished. Script changes and outputs remain under `$RTSS_WORK_DIR/results` on the host.

Local setup. The repository may be placed in `$HOME` by default or another designated directory. Set `RTSS_REPO` to its actual location, then run the scripts from the repository root after activating the Python environment and building `ts_maxp` and `ts_mink`:

```

export RTSS_REPO="${RTSS_REPO:-$HOME/RTSS26-Multi-Telescope-Transient-
Follow-Up-Search}"

# For another location, use this instead:
# export RTSS_REPO="/path/to/RTSS26-Multi-Telescope-Transient-Follow-Up-
Search"

cd "$RTSS_REPO"
conda activate rtss26-figures

```

The compiled executables should be located at:

```

$RTSS_REPO/build/ts_maxp
$RTSS_REPO/build/ts_mink

```

To change an experiment, edit the configuration block near the top of the corresponding Python file. Each script:

- Locates the repository root from its own location.
- Checks the executable, map, and tiling inputs before starting.
- Uses the tiling paired with each map class; tile IDs from different tilings must not be mixed.
- Creates the appropriate directory under `results/`.
- Prints each complete command before executing it.
- Stops immediately if a C++ run returns a nonzero exit status.

Existing CSV files are protected by default. If an output already exists, the script stops before launching a long experiment. Move or rename the old file, or deliberately set:

```
OVERWRITE_RESULTS = True
```

in that script to replace it.

3.2 Common Evaluation Configuration

The paper uses:

- 29 graph instances derived from real LIGO sky-localization maps.
- 15 small instances with 103-218 tiles and 6.9×6.9 degree fields of view.
- 13 large instances with 524-952 tiles and 4.0×2.0 degree fields of view.
- One very large instance with 11,678 tiles and a 1.34×0.9 degree field of view.
- Maximum slew velocity $w_{\max} = 10$ degrees/s.
- Maximum slew acceleration $w_{\text{acc}} = 10$ degrees/s².
- The paper's air-mass-based dwell-time model with reference constant $C_0 = 1$ s.
- The experiment scripts pass `DWELL_ZENITH_SECONDS = 1.0`, which corresponds exactly to $C_0 = 1$ s. They also pass `IS_DEEPSLOW = False` and use the zero-settle-time value compiled into the drivers.
- Random initial telescope roots selected from all-sky tile centers, with the same root set reused across algorithms for each graph instance.

3.3 Run All Experiments

The following subsections describe the complete experiment groups, their output files, the commands that run all groups, and visualization of the generated results.

3.3.1 Fixed- K Cross-Instance Experiments (Figures 2-4)

Configuration:

- $K = 4$ telescopes.
- Route deadline $D = 100$ s.
- Methods: `GREEDY+SA`, `GREEDY+ILP`, `TOP-SA`, and `TOP-ILP`.
- The single-route ILP calls in `GREEDY+ILP` are limited to one hour each.
- `TOP-ILP` is limited to one hour per instance.

Run:

```
cd "$RTSS_REPO"
python3 results/run_maxp_sweep_maps.py
```

The script pairs all `.txt` files in `data/small_maps_6.9x6.9_tiling/` with `data/tilings/6.9x6.9_tiling.csv`, and all files in `data/large_maps_4.0x2.0_tiling/` with `data/tilings/4.0x2.0_tiling.csv`. The `RUN_SMALL_MAPS` and `RUN_LARGE_MAPS` switches can enable either subset.

Results are appended by the C++ driver to:

```
results/max_probability/maxp_small.csv
results/max_probability/maxp_large.csv
```

Expected full runtime on the reference platform: 30+ hours

3.3.2 Fixed- K Deadline Sweeps (Figures 5 and 6)

Configuration:

- $K = 4$.
- Figure 5 uses `data/large_maps_4.0x2.0_tiling/GW191113_071753_952.txt` with D in $\{10, 50, 100, 200, 400, 800\}$ s.
- The Figure 5 ILP limits are one hour for D in $\{10, 50, 100, 200\}$ s, two hours for $D = 400$ s, and ten hours for $D = 800$ s.
- Figure 6 uses `data/very_large_maps_1.34x0.9_tiling/GW200105_162426_11678.txt` with D in $\{200, 400, 800, 1600, 3200, 6400\}$ s.
- Figure 6 reports only `GREEDY+SA` and `TOP-SA`, because ILP methods are impractical at this scale.

Run:

```
python3 results/run_maxp_sweep_deadlines.py
```

Use `RUN_LARGE_MAP_SWEEP` and `RUN_VERY_LARGE_MAP_SWEEP` to run one sweep at a time. The current `ts_maxp` driver still launches its ILP routines on the very-large instance, so the script supplies a 36,000-second safety cap for those additional runs. To execute only the two methods reported in Figure 6, disable the ILP calls in `main_maxp` before building, or use a future method-selection driver.

Results are written to:

```
results/max_probability/maxp_large_map_budget.csv
results/max_probability/maxp_very_large_map_budget.csv
```

Expected full runtime on the reference platform: 25+ hours

3.3.3 Fixed-K Telescope-Count Sweep (Figure 7)

Configuration:

- Instance: large map 191113 with 952 tiles.
- $D = 100$ s.
- $K \in \{1, 2, 4, 8\}$.
- The Gurobi-based methods receive a one-hour limit per configuration.

Run:

```
python3 results/run_maxp_sweep_npaths.py
```

Results are written to:

```
results/max_probability/maxp_large_map_path.csv
```

Expected full runtime on the reference platform: 30+ hours

3.3.4 Minimum-Demand Cross-Instance Experiments (Figures 8-10)

Configuration:

- Route deadline $D = 100$ s.
- Methods: R-GREEDY, R-SA, and R-ILP.
- R-ILP is limited to one hour on small instances and two hours on large instances.
- The candidate-root pool is larger than the number of routes ultimately used, so each method chooses both the roots and the number of routes.

Run:

```
python3 results/run_mink_small.py  
python3 results/run_mink_large.py
```

The small-map script uses the 6.9x6.9 tiling and a 3,600-second R-ILP limit. The large-map script uses the 4.0x2.0 tiling and a 7,200-second R-ILP limit.

Results are written to:

```
results/min_telescope/mink_small.csv  
results/min_telescope/mink_large.csv
```

full runtime on the reference platform: 25+ hours

3.3.5 Minimum-Demand Deadline Sweep (Figure 11)

Configuration:

- Instance: large map 191113 with 952 tiles.
- D in {50, 100, 200, 400, 800} s.
- Methods: R-GREEDY, R-SA, and R-ILP.
- R-ILP is limited to two hours for each deadline.

Run:

```
python3 results/run_mink_sweep_deadlines.py
```

Results are written to:

```
results/min_telescope/mink_large_budget.csv
```

Expected full runtime on the reference platform: 5+ hours

3.3.6 Run Script Summary: Run All Experiment

The following commands execute all experiments sequentially:

```
python3 results/run_maxp_sweep_maps.py
python3 results/run_maxp_sweep_deadlines.py
python3 results/run_maxp_sweep_npaths.py
python3 results/run_mink_small.py
python3 results/run_mink_large.py
python3 results/run_mink_sweep_deadlines.py
```

Expected total runtime on the reference platform: approximately **50 hours or more**, depending on how often the ILP baselines reach their configured time limits.

3.4 Visualize Reviewer-Generated Results

After running any subset of the six experiment scripts, use `results/RTSS2026_Rerun_Results.ipynb` to visualize the newly generated CSV files. This notebook is separate from the precomputed-results notebook and reads exclusively from `results/max_probability/` and `results/min_telescope/`.

For an interactive run from the repository root:

```
conda activate rtss26-figures
jupyter lab results/RTSS2026_Rerun_Results.ipynb
```

The notebook supports partial evaluations. Cross-instance plots contain the available map rows, and sweep plots contain the available deadline or telescope-count values. Figures whose required CSV or representative route instance is missing are reported and skipped without stopping the remaining cells.

Generated PDFs and PNGs are written to:

```
results/paper_figures/
```

3.5 Optional: Reduced or Partial Evaluations

This subsection is optional. Use it only when you do not want to run the complete paper experiments; otherwise, follow Section 3.3.

All reduction controls are in the configuration blocks near the top of the six scripts. Reduced runs exercise the complete C++/Gurobi workflow but do not reproduce the full paper configuration or necessarily match its numerical results.

Note: With a short time limit, an ILP method may return a weak or no feasible solution or a loose/invalid bound

These reduced-run instructions work with both the Docker and local setups. For Docker, edit the scripts in `$RTSS_WORK_DIR/results/` on the host, either before or while the container is running. This directory is mounted as `results/` inside the container, so execute the commands below from the container shell. All script changes, generated CSV files, and figures persist on the host after the container exits.

3.5.1 Maximum-Probability Experiments (`ts_maxp`)

To run only three small maps and three large maps for the fixed-`K` cross-instance experiment, edit `results/run_maxp_sweep_maps.py`:

```
SMALL_MAP_LIMIT = 3
LARGE_MAP_LIMIT = 3
ILP_TIME_LIMIT_SECONDS = 1800 # 30 minutes; use 600 for 10 minutes
```

The script selects the first `N` map filenames in sorted order, making the subset deterministic. Leave either limit as `None` to run every map in that class. `RUN_SMALL_MAPS` and `RUN_LARGE_MAPS` can disable an entire class.

To run fewer deadline values, edit `results/run_maxp_sweep_deadlines.py`. For example:

```
RUN_LARGE_MAP_SWEEP = True
RUN_VERY_LARGE_MAP_SWEEP = False
LARGE_MAP_DEADLINES = [100, 400]
ILP_TIME_LIMIT_OVERRIDE_SECONDS = 1800
```

Set `ILP_TIME_LIMIT_OVERRIDE_SECONDS = None` to recover the paper's deadline-dependent limits. If the very-large-map sweep is enabled, reduce `VERY_LARGE_MAP_DEADLINES` in the same way.

To run fewer telescope counts, edit `results/run_maxp_sweep_npaths.py`:

```
N_PATH_VALUES = [1, 4]
ILP_TIME_LIMIT_SECONDS = 1800
```

Run the selected maximum-probability experiments with:

```
python3 results/run_maxp_sweep_maps.py
python3 results/run_maxp_sweep_deadlines.py
python3 results/run_maxp_sweep_npaths.py
```

3.5.2 Minimum-Telescope Experiments (`ts_mink`)

To run only three instances for each map class, edit both `results/run_mink_small.py` and `results/run_mink_large.py`:

```
MAP_LIMIT = 3
ILP_TIME_LIMIT_SECONDS = 1800 # 30 minutes
```

Leave `MAP_LIMIT = None` to run all maps. As in the maximum-probability runner, the selected subset consists of the first `N` sorted filenames.

To run fewer minimum-demand deadline values, edit `results/run_mink_sweep_deadlines.py`:

```
DEADLINES_SECONDS = [100, 400]
ILP_TIME_LIMIT_SECONDS = 1800
```

Run the selected minimum-telescope experiments with:

```
python3 results/run_mink_small.py
python3 results/run_mink_large.py
python3 results/run_mink_sweep_deadlines.py
```

The minimum-telescope objective does not take a fixed path-count list: the number of routes is the quantity being minimized and is selected internally by `ts_mink`.

3.5.3 Notes for All Reduced Runs

The time-limit value is supplied to the ILP routines within each C++ run; it is not a strict wall-clock limit for the whole Python script. Total time also depends on the number of selected cases and the non-ILP methods.

After editing a script, run it with the same command shown in the relevant subsection of Section 3.3. Reduced outputs use the same `results/` filenames as full runs. If such a file already exists, move it aside first or set `OVERWRITE_RESULTS = True`; do not append a reduced run to an existing full-run CSV. The files under `precompute_results/` are never modified by these scripts.

4. Extending the Artifact

4.1 Run a Single Instance

Reviewers may run an arbitrary instance, whether or not it is part of the paper's evaluation set. `ts_maxp` runs the maximum-detection-probability methods, while `ts_mink` runs the rooted minimum-telescope-demand methods. Each command takes a localization map, its matching tiling, the telescope and experiment parameters, a destination CSV, and a Gurobi time limit.

```
./build/ts_maxp MAP_FILE TILE_FILE \
  DEADLINE W_MAX W_ACC DWELL_ZENITH IS_DEEPSLOW N_PATHS \
  OUT_FILE TIME_LIMIT_SECONDS

./build/ts_mink MAP_FILE TILE_FILE \
  DEADLINE W_MAX W_ACC DWELL_ZENITH IS_DEEPSLOW \
  OUT_FILE TIME_LIMIT_SECONDS
```

The positional arguments have the following meanings:

- `MAP_FILE` and `TILE_FILE` select one localization-map instance and the tiling used to preprocess that map. They must be a matching pair.
- `DEADLINE` is the per-route deadline in seconds.
- `W_MAX` is the maximum angular slew speed in degrees per second.
- `W_ACC` is the angular acceleration magnitude in degrees per second squared. The paper's trapezoidal slew profile uses the same magnitude for acceleration and deceleration.
- `DWELL_ZENITH` is the zenith dwell-time reference constant in seconds; it maps directly to the paper's `C_0`.
- `IS_DEEPSLOW=0` uses great-circle (geodesic) separation between pointings. This is the configuration used in the paper. `IS_DEEPSLOW=1` instead computes motion using separate rotations about the two telescope axes.
- `N_PATHS` is the number of telescopes `K` and is used only by `ts_maxp`.
- `OUT_FILE` is the destination CSV. Its parent directory is created automatically, and the run's results are stored in this file.
- `TIME_LIMIT_SECONDS` is passed to the Gurobi solves. It is not a strict wall-clock limit for the complete executable, which also runs non-ILP computations.

Each executable accepts either the complete positional list shown above or no arguments. Partial argument lists are rejected so that user-supplied values are not silently mixed with compiled defaults. Run

either executable with `--help` to print its interface.

Examples

Maximum-probability driver on a small-map instance:

```
./build/ts_maxp \  
data/small_maps_6.9x6.9_tiling/GW200316_215756_50.txt \  
data/tilings/6.9x6.9_tiling.csv \  
100 10 10 1 0 4 \  
results/max_probability/example_small.csv \  
3600
```

Minimum-demand driver on the representative large-map instance:

```
./build/ts_mink \  
data/large_maps_4.0x2.0_tiling/GW191113_071753_952.txt \  
data/tilings/4.0x2.0_tiling.csv \  
100 10 10 1 0 \  
results/min_telescope/example_large.csv \  
7200
```

4.2 Generate a Different Non-Overlapping Tiling

The planners can be reused to study different tilings. The tiling generator is under `data/generate_sky_tiling/`, and `main.py` is its entry point. The helper modules compute HEALPix footprints, generate rectangular pointings on the sphere, make the pixel-to-tile assignment disjoint, and optionally visualize the result.

The generator additionally requires `healpy`, `astropy`, `astropy-healpix`, and `regions`, which are not part of the base plotting environment. Add them with:

```
conda activate rtss26-figures  
conda install -c conda-forge healpy astropy astropy-healpix regions
```

To make a tiling:

1. Open `data/generate_sky_tiling/main.py` and set `nside`, `width`, and `height` in the `if __name__ == "__main__":` block. `width` and `height` are in degrees.
2. Keep the call to `generate_rectangular_tiling(...)`, which produces the non-overlapping, edge-to-edge rectangular tiling supported by this artifact.
3. Change the filename passed to `write_tile_csv(...)` if desired.
4. Run the generator from its directory:

```
cd data/generate_sky_tiling
python main.py
```

By default, the script writes `tiling.csv` in the current directory. Move the completed file into `data/tilings/` with a descriptive name, then preprocess the intended localization maps against that same tiling before running either scheduler.

4.3 Explore Different Slew Parameters and Separation Models

The existing planners can also be reused to explore telescope dynamics. No rebuild is needed to vary the slew parameters already exposed by the drivers. Change `W_MAX`, `W_ACC`, and `IS_DEEPSLOW` in a Section 4.1 command, or edit the corresponding constants near the top of one of the scripts under `results/`.

- The paper configuration is `W_MAX=10`, `W_ACC=10`, and `IS_DEEPSLOW=0`.
- Changing `W_MAX` varies the maximum slew speed.
- Changing `W_ACC` varies acceleration and deceleration together.
- Changing `IS_DEEPSLOW` from `0` to `1` switches from the paper's geodesic separation to the two-axis separation model.

4.4 Explore Different Dwell-Time Parameters

For the air-mass dwell model used in the paper, the command-line value `DWELL_ZENITH` is exactly the reference constant `C_0` in seconds. The paper configuration is therefore `DWELL_ZENITH=1`. To test another reference exposure, change this positional argument in a Section 4.1 command or edit `DWELL_ZENITH_SECONDS` near the top of the selected script under `results/`; no rebuild is required.

Changing `DWELL_ZENITH` scales the reference exposure while retaining the paper's air-mass relationship. To change the functional form of the dwell model itself, locate its implementation under `include/common/` and `src/common/`, update it, and rebuild the binaries.

For any extension, write results to a new CSV rather than mixing them with the paper configuration, and record the tiling, slew parameters, separation model, dwell constant, deadline, telescope count, and Gurobi time limit used for the run.